

M14 - Kafka + Streaming Foundations

12-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Understand Kafka/event streaming, offsets, replay, delivery semantics and Structured Streaming foundations.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M08 ingestion, M09 reliability, and M12 distributed-processing foundations. You should understand events conceptually, retries, idempotency, and checkpoints.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE14-01 - Batch vs streaming	5
DE14-02 - Events, brokers and topics	7
DE14-03 - Producers and message keys	9
DE14-04 - Consumers and safe processing	11
DE14-05 - Partitions and ordering	13
DE14-06 - Offsets, group position and replay	15
DE14-07 - Consumer groups and scaling	17
DE14-08 - Delivery semantics and idempotency	19
DE14-09 - Serialization, schemas and evolution	21
DE14-10 - Python producer and consumer workflow	23
DE14-11 - Spark Structured Streaming + Kafka	25
DE14-12 - Lag, failures and streaming operations	27

Prerequisite, tool and outcome map

ASSUMED

Complete M08 ingestion, M09 reliability, and M12 distributed-processing foundations. You should understand events conceptually, retries, idempotency, and checkpoints.

MODULE PURPOSE

Understand Kafka/event streaming, offsets, replay, delivery semantics and Structured Streaming foundations.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE14-01 - Batch vs streaming

1. Why it matters

Build and explain the competency: Batch vs streaming. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Batch vs streaming.

3. Prerequisite / what you already know

Complete M08 ingestion, M09 reliability, and M12 distributed-processing foundations. You should understand events conceptually, retries, idempotency, and checkpoints.

4. Picture it in your head

Batch processes a bounded pile of records; streaming repeatedly processes an unbounded sequence as new events arrive. The business question determines acceptable latency and state.

5. New words before we use them

Batch processes bounded data at chosen intervals; streaming processes an ongoing event flow with progress and failure state. Streaming is not automatically faster or better: it adds operational complexity. Choose it when latency and continuous reaction justify the cost.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Classify three requirements as batch or streaming and justify latency/complexity trade-off.

8. Expected result / what you should see

You should be able to build and explain the competency: Batch vs streaming and show a concrete result/state that another person can inspect. Observable target: For Batch vs streaming: Record produced keys/count, consumed count/order for one partition, offset/group position and replay result; confirm duplicate/loss behavior matches the declared delivery rule.

9. Why it worked

The streaming behavior is trustworthy because event identity/order/progress are explicit and the offset/state check can reveal loss, duplication, backlog or unsafe replay. Your independent check is: Do not validate Batch vs streaming by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-01 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Batch processes a bounded pile of records; streaming repeatedly processes an unbounded sequence as new events arrive. 3. Run the earliest independent control: Do not validate Batch vs streaming by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design batch vs streaming choice for payment settlement and defend the latency/SLA boundary.

13. Validate your result

Do not validate Batch vs streaming by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Batch vs streaming in your own words. Start with the purpose: Build and explain the competency: Batch vs streaming. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Batch vs streaming by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Batch vs streaming with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-02.

DE14-02 - Events, brokers and topics

1. Why it matters

Build and explain the competency: Events, brokers and topics. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Events, brokers and topics.

3. Prerequisite / what you already know

Prerequisite: DE14-01 mastered with saved evidence.

4. Picture it in your head

An event is a fact that happened; a broker durably holds event records; a topic is the named log/channel to which producers append and consumers read.

5. New words before we use them

An event is an immutable fact. Producers append events to topics managed by brokers. A topic is split into partitions; each partition is an ordered append-only log with monotonically increasing offsets.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run the normal simulator; inspect topic partitions, events and offsets.

8. Expected result / what you should see

You should be able to build and explain the competency: Events, brokers and topics and show a concrete result/state that another person can inspect. Observable target: For Events, brokers and topics: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The streaming behavior is trustworthy because event identity/order/progress are explicit and the offset/state check can reveal loss, duplication, backlog or unsafe replay. Your independent check is: Do not validate Events, brokers and topics by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-02 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: An event is a fact that happened; a broker durably holds event records; a topic is the named log/channel to which producers append and consumers read. 3. Run the earliest independent control: Do not validate Events, brokers and topics by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design topic/event envelope for parcel status events: key, value, timestamp and retention purpose.

13. Validate your result

Do not validate Events, brokers and topics by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Events, brokers and topics in your own words. Start with the purpose: Build and explain the competency: Events, brokers and topics. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Events, brokers and topics by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Events, brokers and topics with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-03.

DE14-03 - Producers and message keys

1. Why it matters

Build and explain the competency: Producers and message keys. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Producers and message keys.

3. Prerequisite / what you already know

Prerequisite: DE14-02 mastered with saved evidence.

4. Picture it in your head

A producer appends records to a topic. The message key is a routing/identity hint that influences partition choice and therefore ordering for related events.

5. New words before we use them

A producer sends key/value events. The key is not just metadata: partitioning commonly uses it so related events land together. Choose keys from ordering/affinity requirements, not randomly.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Trace each event key to its partition and prove one key never jumps partitions.

8. Expected result / what you should see

You should be able to build and explain the competency: Producers and message keys and show a concrete result/state that another person can inspect. Observable target: For Producers and message keys: Record produced keys/count, consumed count/order for one partition, offset/group position and replay result; confirm duplicate/loss behavior matches the declared delivery rule.

9. Why it worked

The streaming behavior is trustworthy because event identity/order/progress are explicit and the offset/state check can reveal loss, duplication, backlog or unsafe replay. Your independent check is: Do not validate Producers and message keys by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-03 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A producer appends records to a topic. 3. Run the earliest independent control: Do not validate Producers and message keys by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose a message key for account transactions and explain ordering consequence.

13. Validate your result

Do not validate Producers and message keys by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Producers and message keys in your own words. Start with the purpose: Build and explain the competency: Producers and message keys. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Producers and message keys by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Producers and message keys with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-04.

DE14-04 - Consumers and safe processing

1. Why it matters

Build and explain the competency: Consumers and safe processing. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Consumers and safe processing.

3. Prerequisite / what you already know

Prerequisite: DE14-03 mastered with saved evidence.

4. Picture it in your head

A consumer reads records, performs side effects, and advances progress. Safe processing couples “work completed” with offset/state handling so crashes do not silently lose work.

5. New words before we use them

A consumer polls records, performs work, and advances/commits progress. The dangerous moment is between side effect and offset commit: a crash can cause replay. Safe consumers therefore assume reprocessing can happen and make sinks idempotent or coordinate output/progress atomically.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Trace the crash-before-offset-commit scenario and show which event replays.

8. Expected result / what you should see

You should be able to build and explain the competency: Consumers and safe processing and show a concrete result/state that another person can inspect. Observable target: For Consumers and safe processing: Record produced keys/count, consumed count/order for one partition, offset/group position and replay result; confirm duplicate/loss behavior matches the declared delivery rule.

9. Why it worked

The streaming behavior is trustworthy because event identity/order/progress are explicit and the offset/state check can reveal loss, duplication, backlog or unsafe replay. Your independent check is: Do not validate Consumers and safe processing by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-04 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A consumer reads records, performs side effects, and advances progress. 3. Run the earliest independent control: Do not validate Consumers and safe processing by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design a consumer that writes to an external database safely when a crash can occur after write but before offset commit.

13. Validate your result

Do not validate Consumers and safe processing by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Consumers and safe processing in your own words. Start with the purpose: Build and explain the competency: Consumers and safe processing. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Consumers and safe processing by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Consumers and safe processing with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-05.

DE14-05 - Partitions and ordering

1. Why it matters

Build and explain the competency: Partitions and ordering. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Partitions and ordering.

3. Prerequisite / what you already know

Prerequisite: DE14-04 mastered with saved evidence.

4. Picture it in your head

Picture one coordinator with a job board and many workers holding boxes of data. The driver plans; partitions divide data; executors run tasks on those partitions.

5. New words before we use them

Partitions are Kafka's unit of ordering and parallelism. Kafka guarantees order within a partition, not across an entire multi-partition topic. If all events for one customer require order, a stable customer key can keep them together; more partitions increase parallelism but change capacity/operations.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Verify offsets are monotonic inside each partition and explain why global order is not guaranteed.

8. Expected result / what you should see

You should be able to build and explain the competency: Partitions and ordering and show a concrete result/state that another person can inspect. Observable target: For Partitions and ordering: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: Do not validate Partitions and ordering by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-05 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one coordinator with a job board and many workers holding boxes of data. 3. Run the earliest independent control: Do not validate Partitions and ordering by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Given 6 partitions and 9 consumers, predict active/idle consumers and ordering scope.

13. Validate your result

Do not validate Partitions and ordering by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Partitions and ordering in your own words. Start with the purpose: Build and explain the competency: Partitions and ordering. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Partitions and ordering by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Partitions and ordering with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-06.

DE14-06 - Offsets, group position and replay

1. Why it matters

Build and explain the competency: Offsets, group position and replay. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Offsets, group position and replay.

3. Prerequisite / what you already know

Prerequisite: DE14-05 mastered with saved evidence.

4. Picture it in your head

Think key-value and membership: dictionaries answer “what value belongs to this key?” while sets answer “have I seen this unique item?” efficiently.

5. New words before we use them

Offsets identify positions inside one partition. A consumer group commits positions so a restarted member can resume. Resetting to earliest/latest changes what unread history is considered.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use committed_positions and replay list to reconstruct restart behavior.

8. Expected result / what you should see

You should be able to build and explain the competency: Offsets, group position and replay and show a concrete result/state that another person can inspect. Observable target: For Offsets, group position and replay: Record produced keys/count, consumed count/order for one partition, offset/group position and replay result; confirm duplicate/loss behavior matches the declared delivery rule.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: Do not validate Offsets, group position and replay by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-06 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think key-value and membership: dictionaries answer “what value belongs to this key?” while sets answer “have I seen this unique item?” efficiently. 3. Run the earliest independent control: Do not validate Offsets, group position and replay by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write a replay plan from a saved offset without duplicating already-published business effects.

13. Validate your result

Do not validate Offsets, group position and replay by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Offsets, group position and replay in your own words. Start with the purpose: Build and explain the competency: Offsets, group position and replay. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Offsets, group position and replay by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Offsets, group position and replay with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-07.

DE14-07 - Consumer groups and scaling

1. Why it matters

Build and explain the competency: Consumer groups and scaling. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Consumer groups and scaling.

3. Prerequisite / what you already know

Prerequisite: DE14-06 mastered with saved evidence.

4. Picture it in your head

Consumers in one group divide topic partitions among themselves. More consumers help only until you reach the number of partitions available to assign.

5. New words before we use them

Consumers sharing a group divide partitions so one partition is handled by one group member at a time. Adding consumers beyond partition count creates idle members. Membership changes can rebalance assignments, so processing should tolerate handoff/restart and not keep critical state only in one process memory.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect assignment of 3 partitions to 4 consumers and explain the idle member.

8. Expected result / what you should see

You should be able to build and explain the competency: Consumer groups and scaling and show a concrete result/state that another person can inspect. Observable target: For Consumer groups and scaling: Record produced keys/count, consumed count/order for one partition, offset/group position and replay result; confirm duplicate/loss behavior matches the declared delivery rule.

9. Why it worked

The streaming behavior is trustworthy because event identity/order/progress are explicit and the offset/state check can reveal loss, duplication, backlog or unsafe replay. Your independent check is: Do not validate Consumer groups and scaling by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-07 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Consumers in one group divide topic partitions among themselves. 3. Run the earliest independent control: Do not validate Consumer groups and scaling by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Explain what happens during a group rebalance and what local state must not be assumed durable.

13. Validate your result

Do not validate Consumer groups and scaling by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Consumer groups and scaling in your own words. Start with the purpose: Build and explain the competency: Consumer groups and scaling. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Consumer groups and scaling by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Consumer groups and scaling with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-08.

DE14-08 - Delivery semantics and idempotency

1. Why it matters

Build and explain the competency: Delivery semantics and idempotency. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Delivery semantics and idempotency.

3. Prerequisite / what you already know

Prerequisite: DE14-07 mastered with saved evidence.

4. Picture it in your head

At-most-once risks loss, at-least-once risks duplicates, and exactly-once requires coordinated guarantees. Idempotent processing makes duplicate delivery safe at the application boundary.

5. New words before we use them

At-least-once processing can repeat work after failure; at-most-once can lose work; exactly-once claims are scoped. Kafka idempotent producers and transactions can make Kafka-to-Kafka processing strong, but transactions do not magically cover arbitrary external APIs/databases. State the exact boundary of the guarantee.

6. Useful video / source

Source: Confluent Developer — free Apache Kafka courses

Assigned scope: Open the matching Kafka 101 lesson for this concept

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (developer.confluent.io)

URL: <https://developer.confluent.io/courses/>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Compare raw processed+replayed operations with unique idempotent sink event IDs.

8. Expected result / what you should see

You should be able to build and explain the competency: Delivery semantics and idempotency and show a concrete result/state that another person can inspect. Observable target: For Delivery semantics and idempotency: record the actual before/after state and verify the lesson requirement with an independent control.

9. Why it worked

The workflow is trustworthy because progress/state is explicit and replaying the same input is tested to converge without silent loss or duplicate side effects. Your independent check is: Do not validate Delivery semantics and idempotency by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-08 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: At-most-once risks loss, at-least-once risks duplicates, and exactly-once requires coordinated guarantees. 3. Run the earliest independent control: Do not validate Delivery semantics and idempotency by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

State precisely what “exactly once” does and does not guarantee for Kafka→REST API.

13. Validate your result

Do not validate Delivery semantics and idempotency by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Delivery semantics and idempotency in your own words. Start with the purpose: Build and explain the competency: Delivery semantics and idempotency. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Delivery semantics and idempotency by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Delivery semantics and idempotency with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-09.

DE14-09 - Serialization, schemas and evolution

1. Why it matters

Build and explain the competency: Serialization, schemas and evolution. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Serialization, schemas and evolution.

3. Prerequisite / what you already know

Prerequisite: DE14-08 mastered with saved evidence.

4. Picture it in your head

Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. Always know which room and dialect your query is using.

5. New words before we use them

Serialization turns event objects into bytes. A schema makes field names/types/contracts explicit. Evolution needs compatibility rules so old/new producers and consumers can coexist.

6. Useful video / source

Source: Confluent — How to use Data Contracts for Long-Term Schema Management

Assigned scope: 8:38–31:17: Avro/Protobuf, schema references, format changes, forward/backward/full compatibility

How to use it: Use this longer official class for serialization and compatibility thinking. Then complete the Teaching Book schema-evolution task with the video closed.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=ZIJ8cs-cJU)

URL: <https://www.youtube.com/watch?v=ZIJ8cs-cJU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Compare schema v1/v2 and classify additive vs breaking change.

8. Expected result / what you should see

You should be able to build and explain the competency: Serialization, schemas and evolution and show a concrete result/state that another person can inspect. Observable target: For Serialization, schemas and evolution: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

9. Why it worked

The result is trustworthy because structure is treated as a contract rather than inferred casually; the validation checks fields/types/constraints against the declared shape. Your independent check is: Do not validate Serialization, schemas and evolution by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-09 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one PostgreSQL server as a building: databases/schemas organize rooms, tables live inside them, and PostgreSQL syntax is the local language. 3. Run the earliest independent control: Do not validate Serialization, schemas and evolution by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Classify four schema changes under backward/forward compatibility reasoning.

13. Validate your result

Do not validate Serialization, schemas and evolution by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Serialization, schemas and evolution in your own words. Start with the purpose: Build and explain the competency: Serialization, schemas and evolution. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Serialization, schemas and evolution by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Serialization, schemas and evolution with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-10.

DE14-10 - Python producer and consumer workflow

1. Why it matters

Build and explain the competency: Python producer and consumer workflow. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Python producer and consumer workflow.

3. Prerequisite / what you already know

Prerequisite: DE14-09 mastered with saved evidence.

4. Picture it in your head

Picture two simple loops: producer serializes + sends + checks acknowledgement; consumer polls + deserializes + processes + commits progress according to the delivery rule.

5. New words before we use them

A Python Kafka workflow has configuration, producer callbacks/flush, consumer polling/error handling, explicit group ID, offset policy and controlled commits. Credentials belong in environment/config, not source. Save topic, partition, offset and event ID evidence so failures can be diagnosed.

6. Useful video / source

Source: DataTalksClub — real-time streaming workshop

Assigned scope: Producer + consumer sections

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=YDUgFeHQzJU)

URL: <https://www.youtube.com/watch?v=YDUgFeHQzJU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Read producer.py/consumer.py line by line and map every config to behavior.

8. Expected result / what you should see

You should be able to build and explain the competency: Python producer and consumer workflow and show a concrete result/state that another person can inspect. Observable target: For Python producer and consumer workflow: Record produced keys/count, consumed count/order for one partition, offset/group position and replay result; confirm duplicate/loss behavior matches the declared delivery rule.

9. Why it worked

The streaming behavior is trustworthy because event identity/order/progress are explicit and the offset/state check can reveal loss, duplication, backlog or unsafe replay. Your independent check is: Do not validate Python producer and consumer workflow by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-10 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture two simple loops: producer serializes + sends + checks acknowledgement; consumer polls + deserializes + processes + commits progress according to the delivery rule. 3. Run the earliest independent control: Do not validate Python producer and consumer workflow by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Write your own Python producer/consumer skeleton without copying the reference files.

13. Validate your result

Do not validate Python producer and consumer workflow by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Python producer and consumer workflow in your own words. Start with the purpose: Build and explain the competency: Python producer and consumer workflow. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Python producer and consumer workflow by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Python producer and consumer workflow with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-11.

DE14-11 - Spark Structured Streaming + Kafka

1. Why it matters

Build and explain the competency: Spark Structured Streaming + Kafka. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Spark Structured Streaming + Kafka.

3. Prerequisite / what you already know

Prerequisite: DE14-10 mastered with saved evidence.

4. Picture it in your head

Structured Streaming treats Kafka as an incremental source: offsets define input progress, micro-batches/continuous processing update state, and checkpoints make restarts reproducible.

5. New words before we use them

Spark Structured Streaming can use Kafka as a source/sink while applying DataFrame transformations incrementally. Kafka offsets identify source progress; Spark checkpoint state belongs to the streaming query. Do not delete a checkpoint to “fix” a job without understanding that you are changing its recovery identity.

6. Useful video / source

Source: DataTalksClub — real-time streaming workshop

Assigned scope: Stream-processing integration section; Teaching Book maps concept to Spark Structured Streaming

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=YDUgFeHQzJU)

URL: <https://www.youtube.com/watch?v=YDUgFeHQzJU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Read Spark reference code and identify Kafka source options vs checkpoint location.

8. Expected result / what you should see

You should be able to build and explain the competency: Spark Structured Streaming + Kafka and show a concrete result/state that another person can inspect. Observable target: For Spark Structured Streaming + Kafka: Check schema and deterministic row/key/totals first, then inspect partitions/plan/shuffle evidence separately so performance tuning cannot hide a correctness defect.

9. Why it worked

The distributed result is trustworthy because logical row/grain rules are kept separate from physical partitions/execution strategy, then reconciled with deterministic counts/schema/output checks. Your independent check is: Do not validate Spark Structured Streaming + Kafka by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-11 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Structured Streaming treats Kafka as an incremental source: offsets define input progress, micro-batches/continuous processing update state, and checkpoints make restarts reproducible. 3. Run the earliest independent control: Do not validate Spark Structured Streaming + Kafka by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Design Kafka→Spark transformation with stable checkpoint identity and rerun/restart evidence.

13. Validate your result

Do not validate Spark Structured Streaming + Kafka by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Spark Structured Streaming + Kafka in your own words. Start with the purpose: Build and explain the competency: Spark Structured Streaming + Kafka. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Spark Structured Streaming + Kafka by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Spark Structured Streaming + Kafka with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE14-12.

DE14-12 - Lag, failures and streaming operations

1. Why it matters

Build and explain the competency: Lag, failures and streaming operations. This matters because streaming systems must make event identity, order and progress explicit or they can lose, duplicate, delay or reorder business events.

2. What you are learning

Build and explain the competency: Lag, failures and streaming operations.

3. Prerequisite / what you already know

Prerequisite: DE14-11 mastered with saved evidence.

4. Picture it in your head

Consumer lag is the distance between produced and processed offsets. Operations watches lag, error/retry rates, throughput and checkpoint health to distinguish backlog from failure.

5. New words before we use them

Consumer lag is the distance between the latest available offset and a group's committed/processed position. Rising lag can mean slow processing, insufficient parallelism, downstream failures or a stuck consumer. Diagnose lag together with throughput, errors, rebalance history and sink health; do not scale blindly.

6. Useful video / source

Source: DataTalksClub — real-time streaming workshop

Assigned scope: Failure/restart/operational portions + course incident task

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE ([youtube.com](https://www.youtube.com/watch?v=YDUgFeHQzJU))

URL: <https://www.youtube.com/watch?v=YDUgFeHQzJU>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use `lag_by_partition` to identify which partition should be investigated first.

8. Expected result / what you should see

You should be able to build and explain the competency: Lag, failures and streaming operations and show a concrete result/state that another person can inspect. Observable target: For Lag, failures and streaming operations: Confirm row count is unchanged, inspect one partition manually, and verify tie/order behavior with a deterministic secondary sort key when needed.

9. Why it worked

The calculation works because the window defines peer groups/order while preserving row grain; a separate grouped or row-count check confirms that detail rows were not accidentally collapsed or duplicated. Your independent check is: Do not validate Lag, failures and streaming operations by appearance alone. The proof is event/offset/state evidence that can expose loss, duplication, disorder or lag.

10. Common beginner mistakes

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary. DE14-12 - Practice and proof

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Consumer lag is the distance between produced and processed offsets. 3. Run the earliest independent control: Do not validate Lag, failures and streaming operations by appearance alone. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Diagnose rising lag with a five-step evidence-first incident sequence.

13. Validate your result

Do not validate Lag, failures and streaming operations by appearance alone. Use the lesson-specific control: Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. Record the actual observed value/state and compare it with what the task requires. If they differ, the lesson is not yet mastered.

14. Teach it back in your own words

Without notes, explain Lag, failures and streaming operations in your own words. Start with the purpose: Build and explain the competency: Lag, failures and streaming operations. Then name one realistic failure mode and explain how you would prove the result using this check: Do not validate Lag, failures and streaming operations by appearance alone.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Lag, failures and streaming operations with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.